

DepReduce

Automated Dependency Optimization for Artifact-Based Build Systems

Artifact Evaluation

Artifact for "Automated Dependency Optimization for Artifact-Based Build Systems". Reviewers should use the prebuilt Docker archives; image builds are author-only.

The main archive, `depreduce-artifact-issta2026.tar.gz`, contains the tool, examples, paper experiment scripts, paper-project data, and precomputed plots.

Getting Started (Estimated 10 minutes)

```
docker load -i depreduce-artifact-issta2026.tar.gz
docker run --rm -it depreduce-artifact:issta2026
```

Inside the container:

```
cd /artifact/depreduce
depreduce --help
depstat --help
bazel --version
buck2 --version
```

Now, try running DepReduce on the simple C++ Bazel example:

```
cp -a examples/simple-cxx-project /tmp/depreduce-simple-bazel
cat > /tmp/depreduce-simple-bazel/build.sh <<'EOF'
set -e
bazel build //...
EOF
chmod +x /tmp/depreduce-simple-bazel/build.sh

depreduce \
  --build-system bazel \
  -w /tmp/depreduce-simple-bazel \
  -c /tmp/depreduce-simple-bazel/build.sh \
  --config depreduce.toml \
  --output /tmp/depreduce-simple-bazel/logs

ls /tmp/depreduce-simple-bazel/logs
head -5 /tmp/depreduce-simple-bazel/logs/01-attempts.jsonl
```

Expected: `depreduce` completes and creates `00-graph.jsonl` and `01-attempts.jsonl`.

Inspect the build-file edits:

```
git diff --no-index \  
  examples/simple-cxx-project/main/BUILD \  
  /tmp/depreduce-simple-bazel/main/BUILD || true  
  
git diff --no-index \  
  examples/simple-cxx-project/libb/BUILD \  
  /tmp/depreduce-simple-bazel/libb/BUILD || true
```

Expected: `//libb` is removed from `//main:main`, and `//liba` is removed from `//libb:libb`, while the Bazel build still passes.

Reproduce Paper Analyses from Bundled Data

```
python3 scripts/stats/analyze_builds.py  
python3 scripts/stats/analyze_clean_build.py \  
  --data-root data/experiment \  
  --out-dir /tmp/depreduce-clean-plots
```

The precomputed publication plots and clean-build summary are also included:

```
ls plots  
cat plots/clean_build_summary.md
```

Useful data paths: `data/experiment/*/*-output/01-attempts.jsonl`, `data/experiment/*/*-rebuild*.json`, and `data/experiment/*/stats/{incre_build,clean_build}`.

Optional Zirgen Case Study Image (Estimated 2-3 hours)

The optional archive, `depreduce-artifact-zirgen-issta2026.tar.gz`, adds a real-world project case study with [Zirgen](#) checkout at the paper baseline.

```
docker load -i depreduce-artifact-zirgen-issta2026.tar.gz  
docker run --rm -it depreduce-artifact-zirgen:issta2026
```

Inside the container:

```
cd /artifact/zirgen  
git reset --hard df6fb9dda1c20209058d6ee90a8912351b741081  
git clean -fd
```

Prepare the build script:

```
cat > /tmp/zirgen-full-build.sh <<'EOF'
set -e
cd /artifact/zirgen
bazel build //zirgen/dsl:zirgen
bazel test --notest_keep_going //...
EOF
```

Run DepReduce:

```
depreduce \
  --build-system bazel \
  -w /artifact/zirgen \
  -c /tmp/zirgen-full-build.sh \
  --config /artifact/depreduce/scripts/experiments/zirgen.toml \
  --output /tmp/zirgen-full-depreduce-output
```

Summarize the DepReduce log:

```
depstat parse -l /tmp/zirgen-full-depreduce-output/
```

Expected Zirgen summary: `n_removals: 24`, `n_lifting: 10`, `n_flattening: 5`.

It is possible that the numbers differ from the paper due to build flakiness.

Full Experiment Reruns (Not Bundled)

Scripts are under [scripts/experiments](#), [scripts/buck](#), [scripts/rust](#), and [scripts/stats](#). Full real-project reruns require external checkouts, network access, project dependencies, and substantial runtime.

Author Packaging Commands (For Developer Reference; Not for Reviewers)

```
git submodule update --init
docker build -t depreduce-artifact:issta2026 .
docker save depreduce-artifact:issta2026 | gzip -1 > depreduce-artifact-issta2026.tar.gz
sha256sum depreduce-artifact-issta2026.tar.gz > depreduce-artifact-issta2026.tar.gz.sha256

docker build -f Dockerfile.zirgen -t depreduce-artifact-zirgen:issta2026 .
docker save depreduce-artifact-zirgen:issta2026 | gzip -1 > depreduce-artifact-zirgen-issta2026.tar.gz
sha256sum depreduce-artifact-zirgen-issta2026.tar.gz > depreduce-artifact-zirgen-issta2026.tar.gz.sha256
```

Paper Claims Supported by This Artifact

Paper claim	Artifact support	How to evaluate
DepReduce edits dependency declarations and validates changes by rebuilding.	Supported by code and example run.	Run Getting Started; inspect generated <code>01-attempts.jsonl</code> and diffs.
DepReduce supports Bazel, Buck, and Cargo.	Supported by implementation, examples, and scripts.	Run tool help; inspect <code>depreduce/src/supports/</code> and example scripts.
DepReduce performs direct removal, lifting, and flattening.	Supported by reducer code and bundled logs.	Inspect <code>depreduce/src/reducers/</code> and <code>data/experiment/*/*-output/01-attempts.jsonl</code> .
Paper Bazel evaluation and aggregate results.	Supported by paper-project scripts plus available bundled outputs.	Inspect <code>scripts/experiments</code> , <code>data/experiment</code> , and run analysis scripts.
Build-time/action-count analyses.	Supported by bundled build-event logs and plots.	Run <code>scripts/stats/analyze_builds.py</code> and <code>analyze_clean_build.py</code> ; inspect <code>plots/</code> .
DepReduce results on Zirgen.	Supported by optional image with Zirgen checkout, build script, and bundled log.	Run the optional Zirgen section; inspect the summary and compare to paper.

Paper Claims Not Fully Supported by This Artifact

Claim or result	Why it is not fully supported in the packaged artifact
Full rerun of all real-project experiments during review.	Requires external checkouts, network access, project-specific dependencies, and long runtimes.

Recommended Reviewer Scope

Default 30-minute review: run Getting Started and bundled-data analyses.

Deeper (may take hours): load the Zirgen image, run DepReduce, and summarize the bundled Zirgen log.

Overview

Currently supports the following build systems:

- Bazel
- Buck
- Cargo (for Rust)

See [`depreduce`] for implementation details.

How to Use [`depreduce`]

```
depreduce \  
  -w <WORKSPACE_DIR> \  
  -c <BUILD_SCRIPT> \  
  --config <depreduce.toml> \  
  --build-system <bazel|buck|cargo>
```

See scripts in [scripts/experiments](#) or [scripts/buck](#) for more examples.

Example Build Script

```
set -e  
bazel build //...  
bazel test --notest_keep_going -- //...
```

Example `depreduce.toml`

See [depreduce.toml](#)

Extensions to Related Tools

This part is out of the scope of the artifact evaluation or our paper, but we include it here for completeness and to share our efforts in applying related tools to Bazel.

We also extend two related tools to support Bazel build system and investigate their effectiveness on redundant dependency detection. It turns out that neither of them can do it well.

Extending [`mkcheck`] to [`buildfuzz`]

[`buildfuzz`] is the reproduction of the build fuzzing algorithm proposed by [`mkcheck`] (<https://github.com/nandor/mkcheck>) with new features:

1. Use **custom touchers** instead of the `touch` file operation, which will **CHANGE** the file content but not affect the original functionality. 1. Use **SHA256** instead of timestamp to detect file changes. 1. **Restore** touched file content after every round. 1. **Rebuild** the project before every round.

How to Run [`buildfuzz`]

```
buildfuzz --input examples/simple-cxx-project \  
  --artifact examples/simple-cxx-project/bazel-bin \  
  --command buildfuzz/src/test_data/build.sh \  
  --output result_deps.log
```

The result is a JSONL file like below.

```
["a.o",["a.h","a.c"]]
["b.o",["b.h","b.c"]]
["main.o",["main.c","a.h","b.h"]]
["main",["main.o","a.o","b.o"]]
```

Extending [buildfs] to [strace_parser]

[strace_parser] is the reproduction of [buildfs] (<https://github.com/theosotr/buildfs>) with some improvements:

1. **stat / lstat / statfs syscalls were ignored** because we don't know if the accessed file truly exists, and they are mostly used to detect file changes, i.e., usually not a real sign of file consumption. 1. **Syscalls returning -1** will be ignored, because they failed mostly for inexistent files. 1. **clone3 syscall was added** for tracing. Otherwise there will be many missing Newproc operations. 1. **--decode-pids=pidns,comm was added** as the arguments of strace, to resolve the pid within a separate namespace, which is the case of Bazel sandboxing. Otherwise, the pids returned by clone or fork cannot match the pids traced by strace in its own namespace, which prevents us from tracing the process relationship correctly. 1. A **virtual filesystem** was implemented to track symlinks. Bazel creates lots of symlinks because of sandboxing. 1. A **to_link operation was added** to DSL (IR) to support tracking symlinks.

Though [buildfs] claims their approach is applicable to other build systems including Bazel. The fact is, without our efforts, it is really hard to apply it to Bazel. See [Sandboxing - Bazel] for details about the sandboxing mechanism in Bazel.

How to Run strace

```
bazel clean --expunge
bazel shutdown
strace -s 300 \
  -f \
  -e
access,chdir,chmod,chmod,clone,clone3,close,dup,dup2,dup3,execve,fchdir,fchmodat,fchownat,fcntl
l,fork,getxattr,getcwd,lchown,lgetxattr,lremovexattr,lsetxattr,link,linkat,mkdir,mkdirat,mknod
,open,openat,readlink,readlinkat,removexattr,rename,renameat,rmdir,symlink,symlinkat,unlink,un
linkat,utime,utimensat,utimes,vfork,write,writesv \
  --decode-pids=pidns,comm \
  -o strace.log \
  bash ../build_for_strace.sh
```

How to use [strace_parser]

```
strace_parser -i examples/simple-java-project/strace.log -c examples/simple-java-project -o
result_deps.log
```

The result is a JSONL file like below.

```
["a.o", ["a.h", "a.c"]]  
["b.o", ["b.h", "b.c"]]  
["main.o", ["main.c", "a.h", "b.h"]]  
["main", ["main.o", "a.o", "b.o"]]
```

[buildfs]: <https://dl.acm.org/doi/10.1145/3428212> [BuildChecker]:
<https://ieeexplore.ieee.org/document/10981616> [mkcheck]: <https://ieeexplore.ieee.org/document/8812082>
[Skyframe - Bazel]: <https://bazel.build/reference/skyframe> [Sandboxing - Bazel]:
<https://bazel.build/docs/sandboxing> [buildfuzz]: buildfuzz [strace_parser]: strace_parser [depreduce]:
depreduce